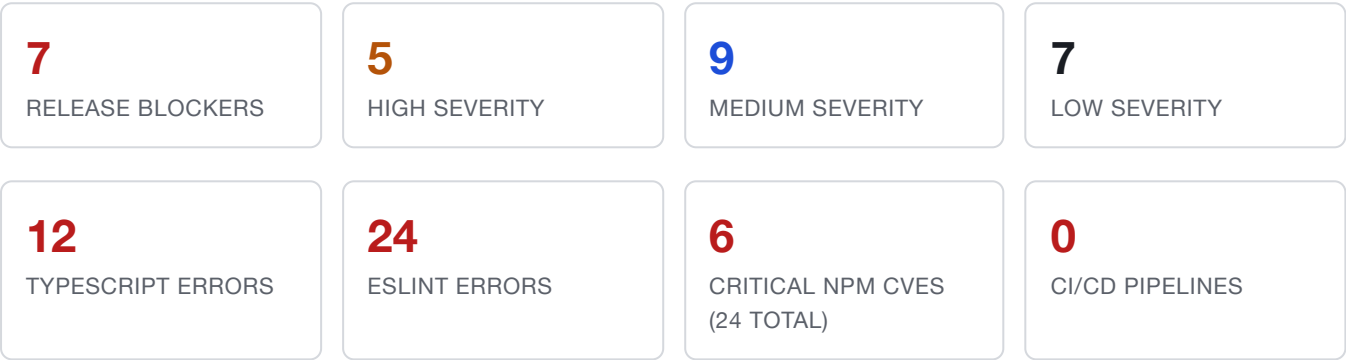


Evolve — Deployment Readiness Report

React Native 0.80.2 + Firebase (Auth, Firestore, Storage, FCM, Cloud Functions) · Android `com.evolve` · iOS (unset)
Audit date: 17 July 2026 · Method: static code review + local quality gates (tsc, ESLint, Jest, npm audit) · Scope: full repository incl. rules & Functions

VERDICT: NOT READY FOR PRODUCTION DEPLOYMENT — 34 / 100



Executive summary

Evolve is a feature-complete robotics/learning app: courses and YouTube playlists, assignments, exams, quiz competitions, projects, events, push notifications, streak/XP gamification, a keyword chatbot, and a full in-app CMS across ~301 TypeScript files and 46 screens. The architecture has genuinely good bones — server-authoritative exam/quiz scoring via Cloud Functions, answer keys isolated in admin-only collections, and a 773-line Firestore ruleset with role helpers and audience targeting.

However, the app **cannot be shipped in its current state**. The admin authorization model is contradictory (UI restrictions are not enforced by rules, and two callable Functions reject superadmins outright, making Notifications and Manage Users features effectively broken). Account deletion violates its own promise to users by orphaning six Firestore subcollections. The codebase fails its own quality gates — TypeScript does not compile, ESLint reports 24 errors, and one Jest suite is broken. There is no CI/CD, no staging environment, no crash reporting, no App Check, and the iOS project still carries the React Native example bundle identifier, which the App Store will not accept.

Every issue below includes a concrete fix. The blocking set is estimated at **1–2 focused weeks**; hardening (Phase 1) at a further 2–3 weeks.

Quality scorecard

Dimension	Score	Ship bar	Assessment
Feature completeness	78	70	Full learner + CMS loop implemented

Architecture	62	70	Sound patterns; scalability gaps in notification fan-out
Security model	38	70	Contradictory admin authz; no App Check; fail-open storage rule
Privacy / compliance	32	70	Incomplete deletion; under-declared privacy manifest; PII in plain storage
Code quality gates	28	70	tsc, ESLint failing; hook-dependency errors are runtime risks
Test coverage	22	70	5 suites, 1 broken; no rules/Functions/E2E tests
Release readiness	18	70	Example iOS bundle ID; no signing team; minify off; dual lockfiles
Ops / observability	15	70	No crash reporting, monitoring, alerting, or staging split
Documentation	25	70	Boilerplate README; unfinished About screen; no runbooks

1. Release blockers (fix before any store build)

BLOCKER SEC-01 Admin UI restrictions are not enforced by Firestore/Storage rules

The UI restricts plain `admin` users to four screens (ManageResources, ManageAssignments, ManageExams, ManageQuizCompetitions) via `adminDashboardAccess.ts` and `AdminFeatureRouteGuard.tsx`. But the Firestore `isAdmin()` helper treats `admin` and `superadmin` identically for nearly every CMS collection. Any plain admin can bypass the UI with a direct Firebase SDK call and mutate courses, notifications, projects, schools, playlists, and more. UI guards are never an authorization boundary.

Evidence: `src/utils/admin/adminDashboardAccess.ts` · `firestore.rules` lines 176–197
(`isAdmin` / `isSuperAdmin`)

FIX

Mirror the UI permission matrix in the rules. Keep `isSuperAdmin()` for full CMS access and restrict `isAdmin()` to the four content types plain admins may manage:

```
// firestore.rules – replace broad isAdmin() usage per-collection
function isContentAdmin(database) {           // admin OR superadmin
    return isAdmin(database);
}
// Collections plain admins may write:
match /resources/{id}      { allow write: if isContentAdmin(database); }
match /assignments/{id}    { allow write: if isContentAdmin(database); }
match /exams/{id}          { allow write: if isContentAdmin(database); }
match /quizCompetitions/{id}{ allow write: if isContentAdmin(database); }
// Everything else in the CMS: superadmin only
match /courses/{id}       { allow write: if isSuperAdmin(database); }
match /notifications/{id}  { allow write: if isSuperAdmin(database); }
match /projects/{id}       { allow write: if isSuperAdmin(database); }
match /schools/{id}        { allow write: if isSuperAdmin(database); }
```

Apply the same split in `storage.rules` for image/PDF upload paths. Add Firestore rules unit tests (see QG-03) asserting a plain admin *cannot* write to `courses`.

BLOCKERSEC-
02**Superadmins are rejected by admin-only Cloud Functions — two features broken**

`sendLiveNotification` (line 279) and `resetUserQuizProgress` (line 397) check `role !== 'admin'` exactly. The screens that call them (Admin Notifications, Manage Users) are superadmin-only in the UI. Result: superadmins open the screens but every server call fails with `permission-denied`; plain admins would pass the server check but cannot reach the screens. Both features are unusable through any supported path. Ironically the file already contains the correct helper — `isAdminRole()` (line 438) accepts both roles but is only used by `migrateAnswerKeys`.

Evidence: `functions/index.js:279, 397, 438` · `src/presentation/screens/Admin/Notifications.tsx` · `ManageUsers.tsx`

FIX

```
// functions/index.js – hoist isAdminRole above the callables and use it
// (decide per-function whether plain admins should also qualify;
// for superadmin-only features, add:)
function isSuperAdminRole(role) { return role === 'superadmin'; }

// sendLiveNotification (matches the superadmin-only UI):
if (!adminSnap.exists || !isSuperAdminRole(adminSnap.data().role)) {
  throw new HttpsError('permission-denied', 'Only superadmins can send live
  notifications.');
```

```
}
```

```
// resetUserQuizProgress: same replacement.
```

Then redeploy: `firebase deploy --only functions`. Add a Functions unit test that calls each callable as `admin`, `superadmin`, and `user` and asserts the expected allow/deny.

BLOCKER PRI-01 Account deletion orphans six Firestore subcollections

`deleteAccountService.ts` deletes the `users/{uid}` document, profile images, and the Auth account — but Firestore never cascades deletes. These subcollections survive forever:

`continueLearningProgress`, `examAttempts`, `quizAttempts`, `fcmTokens`, `notificationPreferences`, `notificationReads`. The UI tells users their associated data is removed; that claim is currently false. This is a GDPR/DPDP right-to-erasure exposure and also leaves live FCM tokens that can still receive pushes.

Evidence: `src/services/firebase/deleteAccountService.ts` (deletes only the parent doc)

FIX

Move deletion server-side where the Admin SDK can delete recursively. Two robust options:

```
// Option A (recommended): Auth-triggered cleanup – also covers console deletions
const functions = require('firebase-functions/v1');
exports.onUserDeleted = functions.auth.user().onDelete(async (user) => {
  const db = getFirestore();
  await db.recursiveDelete(db.doc(`users/${user.uid}`)); // removes all subcollections
});

// Option B: callable invoked before authUser.delete() in deleteAccountService.ts
exports.deleteMyAccountData = onCall(async (request) => {
  if (!request.auth) throw new HttpsError('unauthenticated', 'Sign in first.');
```

Keep the client-side Storage image cleanup, then call the function, then delete the Auth user. Add a test that seeds subcollection docs and asserts none remain after deletion.

BLOCKER QG-01 TypeScript does not compile — 12 errors

`npx tsc --noEmit` fails. Several errors are real behavior bugs, not annotations:

- `MainStack.tsx:66` — `initialState` is not a valid prop on `Stack.Navigator` in React Navigation 7; it is silently ignored, so the "require profile completion" redirect never works via this prop.
- `QuickAccessGrid.tsx:71` — string `width` not assignable to `DimensionValue`.
- `ManageContinueLearningPlaylists.tsx:525, 564` — `autoCorrect` passed to a `FormField` that doesn't accept it.
- `continueLearningPlaylistsService.ts`, `fcmTokenService.ts`, `notificationPreferencesFirestoreService.ts` — error codes `VALIDATION_ERROR`, `PUSH_UNREGISTRATION_ERROR`, `NOTIFICATION_PREFERENCES_SYNC_ERROR` missing from the `FirebaseServiceErrorCode` union; plus an `unknown-to-boolean` assignment.
- `profilePhotoPicker.ts:97` — `0.85` is not a valid `PhotoQuality` (must be one of the allowed literals, e.g. `0.8`).
- `dailyStreakXp.test.ts:1-2` — import paths off by one directory level.

Evidence: `npx tsc --noEmit` output, 17 Jul 2026

FIX

Point fixes in order: (1) pass `initialState` to `NavigationContainer` or use a conditional `initialRouteName` /imperative `navigation.reset()` for the profile-completion redirect; (2) type the `width` as `DimensionValue` (e.g. `'48%'` as `DimensionValue` or compute a number); (3) add `autoCorrect?: boolean` to `FormFieldProps` and forward it to `TextInput`; (4) extend the `FirebaseServiceErrorCode` union with the three missing codes and cast the Firestore read before assigning to `boolean`; (5) change `quality` to `0.8`; (6) fix the two test import paths. Then add `"typecheck": "tsc --noEmit"` to `package.json` scripts and gate CI on it.

BLOCKER QG-02 ESLint fails with 24 errors (44 problems) — several are runtime hazards

The `react-hooks/exhaustive-deps` errors are not cosmetic: `Notifications.tsx:293` omits `handleSendLiveNotification` from a `useCallback` dependency array (stale closure over send logic); `CoursePlaylistScreen.tsx:44` and `CourseVideoScreen.tsx:41` compute objects with `||` expressions that change identity every render, defeating memoization for watch-progress tracking; `ExamAttemptScreen.tsx:99` omits `exam` from a `useEffect`. Plus unused imports (`View`, `TouchableOpacity`, `_removed` ×2) and 20 warnings.

Evidence: `npm run lint` output, 17 Jul 2026

FIX

For each hook error: wrap derived objects in `useMemo` (e.g. `const watchSecondsByVideoId = useMemo(() => progress?.byVideo ?? {}, [progress])`), add the missing dependencies, and verify the affected flows (send notification, exam timer, resume playback) manually afterwards. Delete unused imports. Decide a policy for the `no-void` warnings (either allow `void promise` in `.eslintrc` with `{"allowAsStatement": true}` or replace with explicit `.catch()`). Gate CI on `eslint --max-warnings 0`.

BLOCKER OPS-01 No CI/CD — nothing prevents broken code from shipping

There are no GitHub Actions, no Fastlane, no automated checks of any kind. Given that the repo currently fails tsc, ESLint, and Jest simultaneously, this is not hypothetical — unverified code is already on the main branch. Firestore rules and Functions are deployed by hand with no verification step.

Evidence: no `.github/workflows`, no Fastlane, no pipeline config anywhere in the repo

FIX

```
# .github/workflows/ci.yml — minimum viable gate
name: CI
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npx tsc --noEmit
      - run: npx eslint . --max-warnings 0
      - run: npm test -- --ci
      - run: cd functions && npm ci && npm run lint --if-present
```

Phase 2: add an Android release-build job (`./gradlew bundleRelease` with a debug keystore), Firestore rules tests via the emulator, and Fastlane lanes for store delivery.

BLOCKER OPS-02 iOS release identity is still the React Native example template

The bundle identifier is `org.reactjs.native.example.Evolve` and no `DEVELOPMENT_TEAM` is set. Apple rejects `example` bundle IDs, and the ID is baked into the committed `GoogleService-Info.plist` and Google Sign-In configuration, so changing it late means re-issuing Firebase iOS app config and OAuth clients. Version is 1.0 (build 1) with no versioning scheme.

Evidence: `ios/Evolve.xcodeproj/project.pbxproj:294, 321` · `ios/GoogleService-Info.plist`

FIX

1. Register `com.evolve` (matching Android) in the Apple Developer portal; set `PRODUCT_BUNDLE_IDENTIFIER` and `DEVELOPMENT_TEAM` in both Debug/Release configs.
2. Add a new iOS app with that bundle ID in the Firebase console; replace `GoogleService-Info.plist`; update the reversed client ID in the URL scheme for Google Sign-In.
3. Create an iOS OAuth client for the new bundle ID; verify sign-in end to end.
4. Adopt a version scheme (e.g. marketing 1.0.0, build = CI run number) shared with Android `versionCode / versionName`.

2. High severity

HIGH DEP-01 **24 npm vulnerabilities in the production dependency graph — 6 critical**

`npm audit --omit=dev` (670 prod packages): critical — `@react-native-community/cli` arbitrary OS command injection, `protobufjs` code execution x2, `fast-xml-parser` entity-expansion DoS/bypass, `shell-quote` newline escape, `websocket-driver` x2; high — `@grpc/grpc-js` crash vectors, `ws` memory-exhaustion DoS, `minimatch` and `picomatch` ReDoS; plus 13 moderate, 1 low.

Evidence: `npm audit --omit=dev --json`, 17 Jul 2026

FIX

Run `npm audit fix` first — the majority (`websocket-driver`, `ws`, `yaml`, and others) resolve within semver ranges. For the rest: the CLI advisories are dev-tooling reachable via transitive prod graph — upgrade `@react-native-community/cli` to ≥ 20 alongside the React Native upgrade, or accept with a documented exception since they don't ship in the app binary; `protobufjs` / `grpc-js` come via Firebase Admin in `functions/` — run `npm audit fix` there and bump `firebase-admin` to the latest 13.x. Re-run audit in CI (`npm audit --omit=dev --audit-level=high`) so regressions fail the build. Do *not* use `npm audit fix --force` blindly — it will jump React Native majors.

HIGH SEC-03 **No App Check, rate limiting, or abuse controls on callable Functions**

Any device with a valid Firebase Auth token can invoke `submitExamAttempt`, `submitQuizAttempt`, and the admin callables at unlimited rate from outside the app (the Firebase config in the binary is extractable). There is no App Check attestation, no per-user throttle, and no abuse monitoring — exam submissions can be scripted.

Evidence: `functions/index.js` — no `enforceAppCheck`; no App Check SDK in `package.json`

FIX

```
// 1. Client: add @react-native-firebase/app-check (Play Integrity / App Attest)
// 2. Functions: enforce attestation
exports.submitExamAttempt = onCall({ enforceAppCheck: true }, async (request) => { ...
});
// 3. Simple per-user throttle inside submit callables:
const recent = await db.collection(`users/${uid}/examAttempts`)
  .where('submittedAt', '>', Timestamp.fromMillis(Date.now() - 60_000)).count().get();
if (recent.data().count >= 3) throw new HttpsError('resource-exhausted', 'Too many
attempts.');
```

Roll out in monitor-only mode first, then enforce. Also enable App Check for Firestore and Storage in the Firebase console.

HIGH PRI-02 **iOS privacy manifest declares no data collection — the app collects plenty**

`PrivacyInfo.xcprivacy` claims nothing is collected, while the app stores name, email, phone, school, grade, track, profile photos, FCM tokens, learning progress, and exam/quiz answers. App Store privacy labels must match; a mismatch risks rejection or post-launch enforcement. Android's Play Data Safety form will need the same truthful disclosure.

Evidence: `ios/Evolve/PrivacyInfo.xcprivacy` vs. `users/{uid}` schema and `fcmtokens` subcollection

FIX

Rewrite the manifest with `NSPrivacyCollectedDataTypes` entries for: Name, Email Address, Phone Number, Photos/Videos (profile image), User ID, Device ID (FCM token), Product Interaction (progress/attempts) — each marked linked-to-identity, not used for tracking. Mirror this in App Store Connect privacy labels and the Play Data Safety form. Since this is an education app plausibly used by minors ("kids track"), have counsel review COPPA/GDPR-K obligations before launch. Publish a privacy policy URL (required by both stores) covering the deletion behavior fixed in PRI-01.

HIGH QG-03 **Test suite is thin and partially broken; no tests where the risk actually lives**

Five Jest suites exist; `App.test.tsx` fails because Jest can't transform the ESM build of `@react-native-firebase/messaging` (`jest.config.js` is only `preset: 'react-native'` — no `transformIgnorePatterns` or mocks). Coverage has no `collectCoverageFrom` or thresholds. There are zero tests for Firestore rules, Storage rules, Cloud Functions, auth flows, or account deletion — precisely the areas where this audit found blockers.

Evidence: `jest.config.js` · `__tests__/App.test.tsx` failure · absence of `@firebase/rules-unit-testing`

FIX

```
// jest.config.js
module.exports = {
  preset: 'react-native',
  transformIgnorePatterns: [
    'node_modules/(?!(react-native|@react-native|@react-native-firebase|@react-navigation)/)',
  ],
  setupFiles: ['./jest.setup.js'], // mock @react-native-firebase/* modules here
  collectCoverageFrom: ['src/**/*.ts,tsx', '!src/**/__tests__/**'],
};
```

Then add, in priority order: (1) Firestore rules tests with `@firebase/rules-unit-testing` + emulator, covering the SEC-01 permission matrix; (2) Functions tests for role checks (SEC-02) and recursive deletion (PRI-01); (3) unit tests for gamification/attempt scoring. E2E (Detox/Maestro) can follow post-launch.

HIGH OPS-03 **Single Firebase environment, no crash reporting, no monitoring**

One Firebase project serves development and (future) production — rules experiments and test data hit real users. There is no Crashlytics, no analytics, no performance monitoring, and no alerting. Post-launch, crashes and Function failures would be invisible.

Evidence: `.firebaserc` (single project) · no crash/monitoring SDK in `package.json`

FIX

(1) Create a separate `evolve-staging` Firebase project; add project aliases in `.firebaserc` and use build flavors (Android `productFlavors`, iOS schemes) to select the matching `google-services.json` / plist per environment. (2) Add `@react-native-firebase/crashlytics` and, optionally, `perf` — one install step given the existing RNFirestore setup. (3) In Google Cloud Monitoring, alert on Function error rate and p95 latency for the submit callables. (4) Deploy rules/Functions to staging first in CI, production on tagged release.

3. Medium severity

MEDIUM ARC-01 Notification fan-out scans every user and never cleans up dead tokens

`sendLiveNotification` loads *all* notification preferences and runs a `collectionGroup('fcmTokens')` scan of every token in the database per send, then ignores FCM per-token failures — invalid/rotated tokens accumulate forever, inflating both read costs and failure counts. Fine at 100 users; expensive and slow at 10,000.

Evidence: `functions/index.js` lines 325–367 (`loadNotificationPreferencesByUser` , `collectionGroup('fcmTokens')`)

FIX

Short term: inspect `sendEachForMulticast` responses and delete token docs whose error is `messaging/registration-token-not-registered` or `invalid-argument` . Medium term: switch to **FCM topics** — subscribe devices to `school_{id} / category_{name}` topics client-side on token registration, and publish to topics instead of enumerating tokens. This removes both full-collection scans entirely.

MEDIUM SEC-04 Storage rules accept uploads with missing `contentType` (fail-open)

`storage.rules` validates file type with `!('contentType' in request.resource) || request.resource.contentType.matches(...)` — an upload that omits `contentType` passes. A crafted client could store arbitrary payloads under image/PDF paths.

Evidence: `storage.rules` `contentType` conditions

FIX

```
// Fail closed – require the header and match it:
allow write: if request.resource.contentType != null
    && request.resource.contentType.matches('image/(jpeg|png|webp)')
    && request.resource.size < 5 * 1024 * 1024;
```

MEDIUM SEC-05 One-time migration function `migrateAnswerKeys` remains deployed (*not in earlier summary*)

The callable scans entire exam/quiz collections and rewrites answer-key documents. It is admin-callable (`isAdminRole` , line 816), has served its purpose, and is a standing full-collection write primitive — pure attack/mistake surface.

Evidence: `functions/index.js:816`

FIX

Delete the export and redeploy (`firebase deploy --only functions` removes unlisted functions after confirmation). If migrations may recur, keep the code in a `scripts/` folder run locally with a service account, never as a deployed callable.

MEDIUMSEC-
06**Any authenticated user can read any user's profile image (*not in earlier summary*)**

Storage rules gate profile-image reads on authentication only, not ownership or a legitimate social feature — every signed-in user can enumerate and fetch other users' photos, including minors on the kids track.

Evidence: `storage.rules` `profile-images` path (read: authenticated)

FIX

If profile photos are only shown to the owner, restrict reads to `request.auth.uid == userId` in the path match. If they're shown in shared surfaces (leaderboards), keep authenticated read but document it as intentional in the privacy policy, and ensure image paths are unguessable UIDs (they are — verify no listing rule exists).

MEDIUM

PRI-03

PII and role cached unencrypted in AsyncStorage

`profileCache.ts` stores name, email, phone, school, grade, track, and role in plain AsyncStorage for 10 minutes. AsyncStorage is unencrypted (SharedPreferences / plist-adjacent files); on a compromised or shared device this is readable. The cached *role* also drives UI gating, so a demoted admin keeps admin UI for up to 10 minutes (rules still block server-side).

Evidence: `src/services/profileCache.ts`

FIX

Move the cache to `react-native-keychain` or `react-native-mmkv` with encryption enabled, or reduce cached fields to non-PII (uid, displayName, track). For role freshness, re-read the profile document on app foreground and on admin-screen entry rather than trusting the cache.

MEDIUM

PRI-04

PDFs leak to Google Docs Viewer; WebView has no origin allowlist

`pdfViewerUrl.ts` wraps Firebase Storage download URLs (which embed access tokens) in `docs.google.com/viewer?url=...`, disclosing tokenized URLs to a third party.

`ResourcePdfViewerScreen.tsx` sets no `originWhitelist` or navigation interception, and content URLs are admin-controlled.

Evidence: `src/utils/resources/pdfViewerUrl.ts` · `ResourcePdfViewerScreen.tsx`

FIX

Render PDFs natively with `react-native-pdf` (or `react-native-blob-util` download + native viewer), removing the third-party hop entirely. If the WebView stays for other content, add

`originWhitelist={['https://firebasestorage.googleapis.com']}` and an

`onShouldStartLoadWithRequest` handler rejecting other hosts.

MEDIUM OPS-04 **Build hygiene: dual lockfiles, floating native dependency, minification off**

Both `package-lock.json` and `yarn.lock` are committed — two installers produce different graphs. Android depends on `androidx.activity:activity:1.9.+` (a floating version that changes under you at build time) and release minification/shrinking is disabled, shipping a larger, unobfuscated APK.

Evidence: repo root lockfiles · `android/app/build.gradle` (`1.9.+`, `minifyEnabled false`)

FIX

Pick npm (CI example above uses `npm ci`), delete `yarn.lock`, and add it to `.gitignore`. Pin `androidx.activity:activity:1.9.3`. Enable `minifyEnabled true` + `shrinkResources true` for release with RNfirebase/Reanimated Proguard keep rules, and verify a release build on-device before shipping.

MEDIUM DOC-01 **Documentation is boilerplate; About screen unfinished**

`README.md` is the stock React Native template plus brief auth notes. Missing: Firebase project setup, role model and first-superadmin provisioning, rules/Functions deployment, signing, YouTube key restrictions, staging/prod, data-retention behavior, rollback. `AboutScreen.tsx` renders only the word "About" — a visible-to-users unfinished screen.

Evidence: `README.md` · `src/presentation/screens/About/AboutScreen.tsx`

FIX

Write a real README (setup, environments, deploy commands, role bootstrap via a documented one-time script) plus a short `OPERATIONS.md` (release checklist, rollback, incident contacts). Either finish the About screen (version, licenses, privacy policy link — App Store reviewers look here) or remove it from navigation until done.

MEDIUM ARC-03 **Committed generated iOS artifacts (`ios/.tmp/`) (*promoted from low*)**

Generated React Native codegen output is committed under `ios/.tmp/`. Stale generated code can silently diverge from the installed RN version and confuse Xcode builds after upgrades.

Evidence: `ios/.tmp/` in version control

FIX

`git rm -r --cached ios/.tmp`, add `ios/.tmp/` to `.gitignore`, and rely on `pod install/codegen` to regenerate.

4. Low severity & hygiene (includes items not listed in the earlier summary)

LOW ARC-02 **YouTube API key embedded in the client binary**

`YOUTUBE_API_KEY` via `react-native-config` compiles into the app and is extractable. There is a graceful RSS fallback, so abuse mainly burns quota.

Evidence: `.env.example` · `src/services/youtube/youtubePlaylistService.ts`

FIX

In Google Cloud Console, restrict the key to the YouTube Data API only, bind it to the Android package name + SHA-1 and the iOS bundle ID, and set a daily quota cap. Treat it as public and rotate if quota anomalies appear.

LOW MISC-01 **Placeholder remote assets in production UI (*not in earlier summary*)**

Default/placeholder image URLs remain in `profileDefaults.ts`, `Header.tsx`, and `CourseCard.tsx`. If those third-party hosts throttle or die, avatars and course cards break.

Evidence: `src/constants/profileDefaults.ts` · `src/components/Header.tsx` · `src/components/Home/CourseCard.tsx`

FIX

Bundle default images as local assets (`require('./assets/...')`) or host them in your own Firebase Storage bucket with public-read rules scoped to that folder.

LOW MISC-02 **Hardcoded support contact and OAuth client ID (*not in earlier summary*)**

Support email/phone live in `src/config/support.ts` and the Google web OAuth client ID in `src/config/googleSignIn.ts`. Changing either requires an app release.

Evidence: `src/config/support.ts` · `src/config/googleSignIn.ts`

FIX

Acceptable at launch. Post-launch, move support contact into Firebase Remote Config so it's updatable without a release. The OAuth client ID is public by design — just confirm it matches the production Firebase project after OPS-02.

LOW MISC-03 **Missing package scripts for routine gates (*not in earlier summary*)**

No `typecheck`, `test:coverage`, `release-build`, or `rules-deploy` scripts — every gate is invoked ad hoc, which is why they drifted into failure unnoticed.

Evidence: `package.json` scripts block (5 scripts total)

FIX

```
"scripts": {
  "typecheck": "tsc --noEmit",
  "lint": "eslint . --max-warnings 0",
  "test": "jest",
  "test:coverage": "jest --coverage",
  "verify": "npm run typecheck && npm run lint && npm test",
  "android:release": "cd android && ./gradlew bundleRelease",
  "deploy:rules": "firebase deploy --only firestore:rules,storage"
}
```

LOW MISC-04 **No startup validation of required configuration (*not in earlier summary*)**

A build with a missing/blank `YOUTUBE_API_KEY` or mismatched Firebase config fails silently at feature-use time rather than loudly at startup or build time.

Evidence: no config assertion in `src/App.tsx` or build scripts

FIX

Add a small `assertConfig()` called at app bootstrap in dev builds that warns on missing env values, and a CI step that checks `.env` keys against `.env.example`.

LOW MISC-05 **Committed Firebase client config — verify key restrictions (*not in earlier summary*)**

`google-services.json` and `GoogleService-Info.plist` are committed. Firebase client keys are not secrets, but unrestricted keys allow quota abuse against your project.

Evidence: `android/app/google-services.json` · `ios/GoogleService-Info.plist`

FIX

In Google Cloud Console → Credentials, restrict each auto-created API key by application (Android package + SHA-1, iOS bundle ID) and by API. Combined with App Check (SEC-03) this closes the abuse vector. No need to purge from git.

LOW MISC-06 **Unrestricted device orientations on iOS (*not in earlier summary*)**

iOS supports portrait plus both landscapes while the layouts are designed portrait-first; several screens (forms, admin CMS) will render awkwardly rotated.

Evidence: `ios/Evolve/Info.plist` supported orientations

FIX

Lock to portrait (remove landscape entries) unless video playback needs landscape — in that case allow rotation only on the video screen via a per-screen orientation lock.

5. Remediation roadmap

Phase 0 — Unblock (1–2 weeks) · items: SEC-01, SEC-02, PRI-01, QG-01, QG-02, OPS-01, OPS-02

1. Fix the two Functions role checks (SEC-02) — smallest change, restores two broken features. Redeploy Functions.
2. Align Firestore/Storage rules with the UI permission matrix (SEC-01). Deploy rules.
3. Add the recursive-deletion trigger/callable (PRI-01). Deploy and test with seeded data.
4. Clear all 12 TypeScript errors and 24 ESLint errors (QG-01/02); manually verify notification send, exam timer, and resume-playback flows touched by hook fixes.
5. Land the CI workflow (OPS-01) so the gates can never silently regress again.
6. Set the production iOS bundle ID, team, and Firebase iOS app (OPS-02).

Phase 1 — Harden (2–3 weeks) · items: DEP-01, SEC-03...06, PRI-02...04, QG-03, OPS-03/04, DOC-01

1. `npm audit fix` both packages; pin remaining advisories with documented exceptions; add audit to CI.
2. App Check in monitor mode → enforce; per-user throttles on submit callables.
3. Fail-closed Storage contentType; scope profile-image reads; remove `migrateAnswerKeys`.
4. Rewrite privacy manifest + store data-safety forms; publish privacy policy; native PDF rendering.
5. Jest config fix + rules/Functions test suites; coverage thresholds.
6. Staging Firebase project, Crashlytics, Function alerting; single lockfile, pinned androidx, minified release build.
7. Real README + operations runbook; finish or hide the About screen.

Phase 2 — Operate (post-launch)

1. FCM topic-based fan-out and dead-token cleanup (ARC-01).
2. Encrypted local cache / reduced PII footprint (PRI-03).
3. E2E smoke tests (Maestro/Detox) on physical devices; Fastlane store delivery.
4. Remote Config for support contact and feature toggles; planned React Native 0.8x→latest upgrade window.

6. Ship gate checklist

Gate	Criterion
Compile	<code>tsc --noEmit</code> and <code>eslint . --max-warnings 0</code> pass in CI
Tests	All Jest suites green; Firestore rules tests cover the admin permission matrix
Authorization	Plain admin cannot write non-delegated collections (rules test); superadmin can use Notifications & Manage Users end-to-end

Privacy	Account deletion leaves zero user documents (verified with seeded data); manifest & store labels match collected data; privacy policy live
Dependencies	No unresolved critical/high advisories in <code>npm audit --omit=dev</code>
Identity	Production bundle IDs on both platforms; release signing verified; versioning scheme in place
Ops	Crashlytics reporting from a release build; staging project receives deploys before production

Evolve deployment readiness report · generated 17 July 2026 · based on static analysis and locally executed quality gates (tsc, ESLint, Jest, npm audit) at commit state as of audit time · this is not a penetration test and no live production Firebase project was probed.